

PRD — On-device Visual Perception & Privacy Gateway for Browser Agents

2026-09-22 · @Someone

1. Overview and primer

We are building a browser extension plus a server that together let an AI agent see and operate a web page for the user, without any personal data leaving the user's machine. The extension does all perception and privacy filtering locally; the server only ever receives a sanitized, structured description of the screen and returns structured actions.

1.1 The problem in plain words

An "agent" is an AI that does tasks for you: fill a form, find a setting, book a slot. To do that on a website, it must know what is on the screen and then click or type. Today most agents send a full screenshot and the page's HTML to a cloud model. That screenshot can contain your bank balance, your face on a video call, your password field, your Aadhaar number.

This product flips the default. The browser itself looks at the page, builds a map of what is there, removes everything personal, and sends only the cleaned map to the server. The server decides what to do and replies with commands like "click element e14". The browser carries them out.

1.2 Why this is now possible

Browsers can now run small neural networks on the user's own GPU or CPU. WebGPU gives JavaScript direct GPU access; WebAssembly (WASM) runs compiled code at near-native speed. Libraries such as ONNX Runtime Web and Transformers.js load exported models and run them through those backends. A 5–30 MB model can now process a screenshot in tens of milliseconds on a normal laptop.

1.3 The core design idea

The product has one rule that everything else serves: **raw pixels and raw DOM never cross the network.** Perception, OCR, PII detection and redaction happen on the client. The server is a reasoning engine over clean, structured text, and it is told exactly which

fields were redacted and how. Heavy visual reasoning on the server is an exception path, used only for small redacted image crops when local perception is not confident.

1.4 Glossary

Term	Meaning in this document
DOM	The browser's live tree of page elements (inputs, buttons, text).
Accessibility (AX) tree	A simplified DOM the browser builds for screen readers: roles, names, states.
PII	Personally identifiable information: names, emails, phone, IDs, card numbers, faces, addresses.
Redaction	Removing or masking sensitive content (blackout, blur, placeholder token).
Placeholder token	A stand-in like <code>{{EMAIL_1}}</code> sent instead of the real value; the real value stays in a local vault.
Vault	In-memory client store mapping placeholders to real values for one session.
ViT	Vision Transformer, an image model that splits images into patches. Here used broadly for any local vision model.
Bounding box (bbox)	Rectangle <code>[x, y, w, h]</code> in page pixels locating an element or region.
Screen State	The sanitized, structured JSON snapshot of a page that the client sends to the server.
Action	A structured command from the server (click, type, scroll, wait, ask_user, done).
Tier 0/1/2	Escalation levels: DOM-only, local vision/OCR added, redacted crop sent to server VLM.
VLM	Vision-language model: an LLM that also accepts images.

Term	Meaning in this document
Constrained decoding	Forcing an LLM to emit output that matches a JSON schema or grammar.
MV3	Manifest V3, the current browser extension platform for Chrome and Firefox.
Offscreen document	A hidden extension page in Chrome MV3 that can use DOM and GPU APIs the service worker cannot.

2. Goals, non-goals and success metrics

The product succeeds when an agent completes real multi-step web tasks with zero PII egress, under 2.5 s median per step, on a mid-range laptop.

2.1 Goals

1. Understand the current screen locally: every actionable element, its role, label, state and position.
2. Detect and redact PII locally in both the structured data and any image crop, before any network call.
3. Send the server a compact, redaction-aware Screen State; receive schema-valid actions.
4. Execute actions reliably in the page, verify the result, and loop until the task is done or needs the user.
5. Run in Chrome and Firefox (desktop) as one codebase.
6. Stay within a strict client resource budget and degrade gracefully on weak hardware (WebGPU → WASM).
7. Offer four modes on one stack: Ask, Guide (point and draw, HeyClicky-style), Agent, and Background (Comet-style), with local voice input and output.

2.2 Non-goals (v1)

- Mobile browsers and Safari.
- Operating outside the browser tab (desktop apps, OS dialogs).
- Solving CAPTCHAs or bypassing anti-bot measures.
- Fully autonomous payments or irreversible actions without user confirmation.

- Training large models from scratch; we fine-tune and distil small ones.

2.3 Success metrics and targets

The five evaluation criteria map directly to product KPIs. Targets are v1 launch gates on the internal benchmark (Section 12).

Criterion (weight)	KPI	How measured	v1 target
Visual context accuracy (25%)	Element recall and label accuracy vs ground truth	Matched elements (IoU ≥ 0.5) / all actionable elements; exact-role + normalized-label match	Recall $\geq 95\%$, role acc $\geq 93\%$, label acc $\geq 90\%$
PII detection (20%)	Precision and recall per PII class	Detected sensitive spans/regions vs annotated ground truth	Recall $\geq 98\%$ (passwords/cards/IDs 100%), precision $\geq 90\%$
Redaction precision (20%)	Pixel and text redaction quality	Pixel IoU of mask vs GT; over-redaction rate of non-PII	Mask IoU ≥ 0.85 , leaked pixels 0 for high-risk classes, over-redaction $\leq 5\%$
Client resources (20%)	CPU, GPU, RAM, main-thread blocking	Chrome performance traces on reference laptop	Peak extra RAM ≤ 350 MB, model download ≤ 40 MB, main-thread long tasks ≤ 50 ms
End-to-end latency (15%)	Time per agent step and per task	Capture start $\rightarrow$ action executed	Step p50 ≤ 1.5 s, p95 ≤ 3.0 s (Tier 0); task p50 ≤ 20 s on 5-step tasks

2.4 Guardrail metrics

- **Known-PII egress on the benchmark and canary suites: 0.** Any seeded PII string, known vault value, or unmasked high-risk pixel region reaching the network fails the build. This is a test result on known data, not a claim that no personal information can ever escape; the product's guarantee is the fail-closed policy in Section 6.7.

- **Schema-invalid server responses: < 0.5%** (target 0 with constrained decoding).
- **Wrong-element execution rate: < 2%** of actions.
- **Unknown-outcome replays: 0.** Consequential actions with an unknown outcome are never retried automatically (Section 9.3).

2A. Product experience: guide mode meets agent mode

The extension combines HeyClicky's summon-by-hotkey, talk-out-loud, draw-on-screen tutor with Comet's page- and tab-aware sidecar agent, and differs from both by keeping perception and PII on the device. It stays a Chrome/Firefox extension, not a browser fork.

2A.1 What we take from each product

Source	Feature	How it appears in our extension	Change vs original
HeyClicky	Hotkey-summoned; only looks at the screen when invoked	Global shortcut (e.g. Alt+Space) and push-to-talk; perception runs only on invoke or during an active task	Same principle, but scoped to the browser tab
HeyClicky	Talk out loud; it walks you through what you are stuck on	Voice conversation about the current page	Speech recognition runs locally; raw audio never leaves the device
HeyClicky	Draws on screen to point the way	Overlay layer: animated pointer, circles, arrows, step badges anchored to elements	Anchored to element IDs, so drawings follow scroll and layout changes
HeyClicky	Agent mode does the task for you	Agent mode (Section 9 loop)	Same, with local redaction and confirmations
HeyClicky	Friendly on-screen buddy	Small floating companion near the cursor that shows state (listening, thinking, acting)	Cosmetic; can be turned off

Source	Feature	How it appears in our extension	Change vs original
Comet	Sidecar panel that reads the current page, summarises, answers	Side panel "Ask" mode over the sanitized page	Comet processes page content on its servers; we send only redacted text
Comet	Chat with your tabs; context across open tabs	Multi-tab digests (Section 3.4) the user can reference: "compare these three tabs"	Digests are built and redacted locally
Comet	Interactive agent you watch and can interrupt; voice feedback mid-task	Live action trace, pause/stop, spoken corrections ("no, the other button")	Same
Comet	Background assistants on a schedule	Background tasks in a separate window, triggered manually or by schedule (v1.5)	High-risk steps queue for approval instead of running unattended
Comet	Asks permission before acting in the browser	Per-task permission grant plus per-action confirmation for high-risk actions	Stricter: permission is per site, per task

2A.2 The four modes

Mode	User intent	Who clicks	What the server returns	Latency target	Default risk
Ask	"What does this page say? What is this setting?"	Nobody	Answer text (placeholders resolved locally)	First token ≤ 1.2 s	None
Guide	"Show me how to change my phone number"	The user	<code>point</code> / <code>annotate</code> steps with short spoken instructions	Step ≤ 1.5 s	None: the agent never acts
Agent	"Change my phone number to ..."	The agent	Actions (Section 7.3)	Step ≤ 1.5 s p50	Confirm high-risk

Mode	User intent	Who clicks	What the server returns	Latency target	Default risk
Background	"Every morning, check my exam portal for new notices"	The agent, unattended	Actions + final report	Not interactive	Read-only by default; writes need prior approval

Guide mode is the recommended default for new users and sensitive sites. It uses the same perception and planning stack as Agent mode, but the executor draws instead of acting. This makes it safer (no state change) and a natural way to build trust before the user delegates.

2A.3 Key user flows

1. **Guide:** user presses the hotkey and says "how do I download my fee receipt?" → local STT → PII scan of the transcript → Screen State → planner returns step 1 point e31 "Open the Payments tab" → overlay pulses on the tab and TTS speaks → extension detects the user's click (page change) → next step, until done.
2. **Agent with voice correction:** user says "fill this form with my details" → agent types vaulted values → user says "wait, use my college email" → transcript becomes a `user_interrupt` event in the next step → planner revises.
3. **Ask across tabs:** user has three laptop listings open and asks "which is cheapest with 16 GB RAM?" → each tab's digest (already sanitized) is sent → answer with links back to tabs; clicking a citation focuses that tab and highlights the element.
4. **Background:** user schedules a daily check → at the scheduled time the extension opens the site in a minimised window, runs Agent mode read-only, and posts a notification with the result; any write action waits in an approval queue.

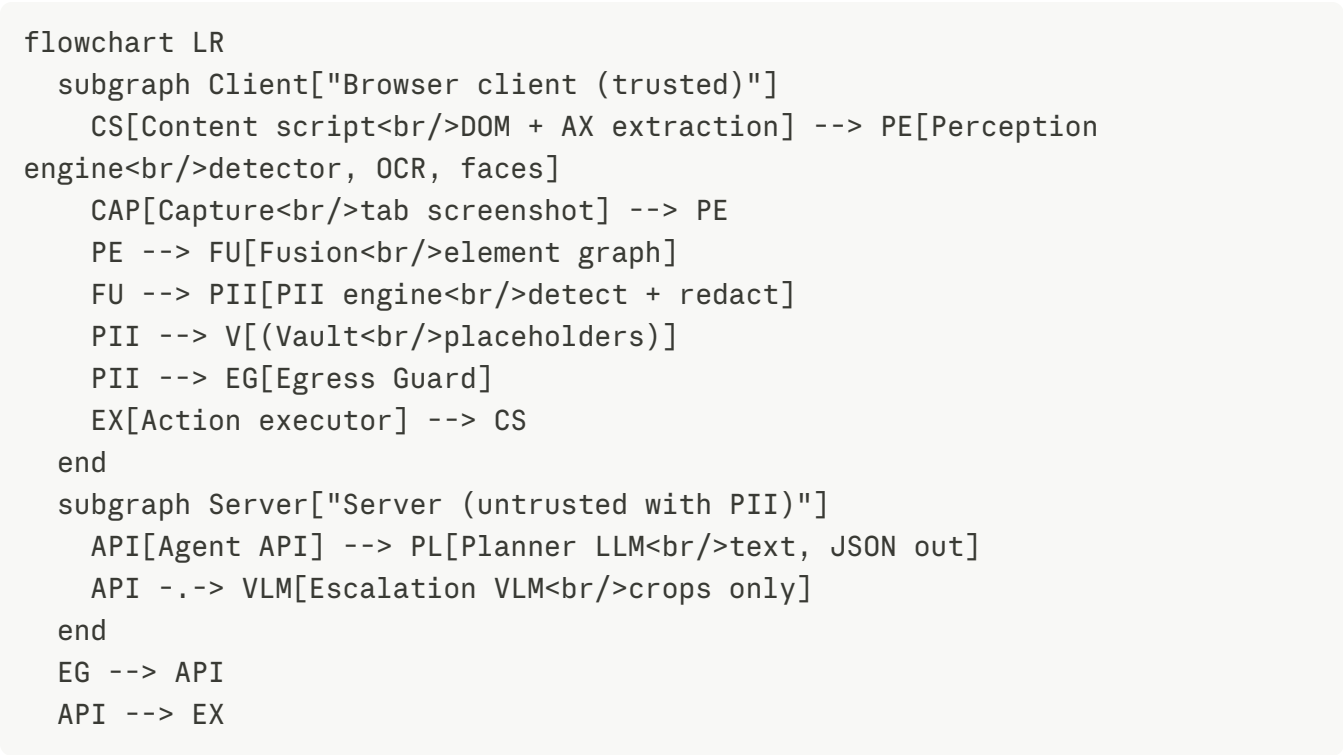
2A.4 Explicit non-goals from these products

- Screen-wide assistance outside the browser (HeyClicky works across all Mac apps); we stay in-tab.
- Server-side speech-to-speech voice models: they need raw audio on the server, which breaks the privacy rule.
- Connected-account agents (email, calendar APIs) in v1; the agent works only through pages the user has open.
- Building our own browser; Comet is a Chromium fork, we are an extension.

3. System architecture

The system is split by a hard trust boundary: the client owns all raw data and all perception; the server owns planning over sanitized text. A single Egress Guard is the only code path allowed to make network calls.

3.1 Components



Solid arrows are the default path; the dotted arrow is the rare Tier 2 escalation.

Component	Runs in	Responsibility
Content script	Page context (isolated world)	Walk DOM and AX info, assign stable element IDs, execute actions, observe mutations
Capture	Extension background	Take viewport screenshot (<code>captureVisibleTab</code>) on demand
Perception engine	Offscreen document (Chrome) / background page (Firefox), inside a Web Worker	Run local models on WebGPU or WASM
Fusion	Worker	Merge DOM elements and visual detections into one element graph

Component	Runs in	Responsibility
PII engine	Worker	Classify sensitive elements, spans and regions; apply redaction; fill vault
Vault	Worker memory only	Placeholder → real value map; never serialized to network or disk
Egress Guard	Background	Final scan of every outbound payload; blocks on any violation
Action executor	Content script	Resolve element IDs and placeholders, perform DOM actions, verify
Agent API	Server	Session state, prompt assembly, validation, routing
Planner LLM	Server GPU	Reason over Screen State, emit one action per step
Escalation VLM	Server GPU	Interpret small redacted crops when local confidence is low

3.2 Tiered perception (escalation ladder)

The cheapest sufficient signal is always used first. The client decides the tier per element, not per page.

Tier	When used	Client work	What is sent	Expected share of steps
0: Structural	Element has usable DOM/AX text and role	DOM walk, visibility check, text PII scan	Text-only Screen State	~80%
1: Local visual	Element is visually present but DOM is poor (canvas, image, icon-only button, custom widget)	UI detector, OCR on that region only, icon classifier	Text-only Screen State with vision-derived fields	~17%

Tier	When used	Client work	What is sent	Expected share of steps
2: Server visual	Local confidence below threshold, or task needs visual judgment (chart, map)	Crop region, redact crop, downscale	Screen State + ≤ 2 redacted crops (≤ 512 px long side)	~3%

3.3 One agent step, end to end

```
sequenceDiagram
    participant U as User
    participant CS as Content script
    participant W as Worker (perception + PII)
    participant EG as Egress Guard
    participant S as Server
    U->>CS: Task: "Update my phone number"
    CS->>W: DOM snapshot + screenshot
    W->>W: Fuse, detect PII, redact, fill vault
    W->>EG: Screen State (sanitized)
    EG->>S: POST /v1/step
    S->>S: Planner LLM, constrained JSON
    S-->>CS: Action: type e14 {{PHONE_NEW}}
    CS->>W: Resolve placeholder
    CS->>CS: Execute, wait for settle
    CS->>W: Next snapshot (diff)
```

The loop repeats until the server returns `done` or `ask_user`. Only changed elements are re-sent after the first step.

3.4 Assistant-layer components

Five client components sit on top of the perception and privacy core to deliver the modes in Section 2A. None of them adds a new network path; everything still leaves through the Egress Guard.

Component	Runs in	Responsibility	Key implementation
Invocation manager	Background + content script	Hotkey (<code>commands</code> API), push-to-talk, mode selection, per-task site permission	Perception is idle until invoked; permission prompt on first use per site
Voice pipeline	Offscreen document / worker	Mic capture, voice activity detection, local speech-to-text, text-to-speech	See 3.4.1
Overlay renderer	Content script, closed shadow root	Draws pointer, circles, arrows, step badges, companion; highlights agent targets	See 3.4.2
Tab orchestrator	Background	Tracks open tabs, builds sanitized per-tab digests, runs background tasks	See 3.4.3
Local memory	Worker + IndexedDB (encrypted)	Short sanitized notes about past tasks ("user's portal is X")	Stores placeholder-level text only; user can view and wipe

3.4.1 Voice pipeline

1. **Capture:** `getUserMedia` in the offscreen document (Chrome) or background page (Firefox); mic permission granted once via an extension page.
2. **Voice activity detection:** Silero VAD (ONNX, ~2 MB) to segment speech and detect barge-in while TTS is speaking.
3. **Speech-to-text:** Whisper tiny or base (English + Hindi) or a Moonshine-class model via `Transformers.js` on WebGPU; ~40–75 MB extra, lazy-loaded only when voice is first used. Target: final transcript ≤ 500 ms after end of speech for a 5 s utterance on WebGPU.
4. **Do not use** the browser Web Speech recognition API: in Chrome it sends audio to a cloud service, which violates the privacy rule.
5. **Transcript privacy:** transcripts go through the PII engine like page text; spoken values ("my number is ...") are vaulted before anything is sent.

6. **Text-to-speech:** browser `speechSynthesis` restricted to local voices (`voice.localService === true`); optional Piper/Kokoro-class WASM voice for a consistent sound.
7. **Streaming:** the planner's spoken `say` field streams via SSE, so TTS starts before the full response arrives.

3.4.2 Overlay renderer

- Mounted in a closed shadow root at the top of the page with `pointer-events: none` and `all: initial` styles. The shadow root gives style encapsulation, not security: the page can still cover, imitate or visually observe the overlay. So the overlay only shows guidance; secrets, approvals and confirmations live in extension-owned UI (side panel), never in the page. User clicks pass through.
- Annotations are anchored to element IDs, not fixed coordinates. A `ResizeObserver`, scroll listener and `requestAnimationFrame` loop re-position them when the layout moves.
- Primitives: `pointer` (animated cursor that glides to the target), `circle`, `arrow`, `underline`, `badge` (step number), `label` (short text bubble), `spotlight` (dims everything except the target).
- Guide mode waits for the user's interaction on the pointed element (click, input, or navigation) before advancing, detected via the content script's event listeners.
- The companion is a small SVG/CSS character near the cursor with states: idle, listening, thinking, acting, needs approval.

3.4.3 Tab orchestrator

- Maintains a digest per tab the user has granted: title, sanitized URL, landmarks, top headings, key facts, actionable elements summary (~300–800 tokens), built locally from the same perception output and re-built on change.
- "Chat with tabs" sends only the digests of tabs the user selects (default: current window, max 8), never full pages.
- Background tasks run in a separate minimised window created with `windows.create({state: "minimized"})`; `alarms` API schedules them. Each task has an allowlist of sites and a read-only flag. Creating a task requests optional host permission for every allowlisted site; `activeTab` does not cover sites opened later by a schedule.
- Firefox has no minimised-window guarantee for extensions in all environments; fallback is a background tab in the current window.

4. Client: extension, capture and DOM extraction

The client is one TypeScript MV3 extension built for Chrome and Firefox from a shared core, with browser-specific shims only for background execution and screenshot capture.

4.1 Extension structure

Part	Chrome MV3	Firefox MV3	Notes
Background	Service worker (no DOM, can be killed after ~30 s idle)	Event page (background script with DOM)	Keep it thin: routing, capture, Egress Guard
ML host	Offscreen document (<code>chrome.offscreen</code> , reason <code>WORKERS</code>) hosting a Web Worker	Worker spawned from background page	The Chrome service worker can be stopped when idle, which would drop loaded models, and it cannot use <code>getUserMedia</code> or DOM APIs; the offscreen document gives a stable host. WebGPU itself runs in the dedicated worker
Page access	Content script, isolated world	Same	Injected on demand via <code>scripting.executeScript</code>
UI	Side panel (<code>chrome.sidePanel</code>)	Sidebar (<code>sidebar_action</code>)	Task input, live plan, confirmations, privacy view
Storage	IndexedDB for model weights and settings	Same	Vault is never persisted

Permissions are split by feature, because `activeTab` only grants temporary access to the tab the user invoked the extension on. It does not cover other tabs, cross-origin frames inside that tab, or sites opened later by a scheduled task.

Feature	Permission needed	How it is obtained
Current tab, on invoke	<code>activeTab</code> , <code>scripting</code>	Hotkey, toolbar click or side panel action; lapses when the tab navigates to another origin

Feature	Permission needed	How it is obtained
Cross-origin iframes in the current tab	Host permission for each frame origin (<code>all_frames</code> only selects frames, it grants nothing)	<code>optional_host_permissions</code> requested at runtime, per origin, with a prompt naming the site; otherwise the frame is reported as <code>opaque_frame</code> and handled in Guide mode
Additional tabs for "chat with tabs"	Host permission for each tab's origin	User picks tabs in the side panel; the extension requests those origins
Background tasks	Host permission for every site in the task's allowlist	Requested when the task is created; the task cannot run without it
Core	<code>storage</code> , <code>alarms</code> , <code>offscreen</code> (Chrome), <code>sidePanel</code> (Chrome) / <code>sidebar_action</code> (Firefox)	Install time

`<all_urls>` is never requested. Users can revoke any granted origin from the settings page.

Cross-browser code uses `webextension-polyfill` and a small `platform/` adapter with two implementations. Build with Vite + `@crxjs/vite-plugin` (Chrome) and `web-ext` (Firefox).

4.2 Capture layer

1. On each step, the content script first produces the DOM snapshot (Section 4.3). Screenshot capture is **conditional**: only if Tier 1 or Tier 2 elements exist, or on the first step of a page for layout verification.
2. Screenshot: the background calls `tabs.captureVisibleTab(windowId, {format: "jpeg", quality: 80})`, which returns a data-URL string. Chrome rate-limits this to about 2 calls per second; the scheduler respects that.
3. **Transport, hop 1 (background → offscreen document)**: extension messaging JSON-serializes messages, so an `ImageBitmap` cannot be sent this way. The data URL (typically 150–400 KB) is sent as a string, or the offscreen document fetches it from a short-lived in-memory handoff. Firefox skips this hop because the background page hosts the worker directly.

4. **Decode:** the offscreen document (or Firefox background page) converts the data URL to a `Blob` and calls `createImageBitmap`.
5. **Transport, hop 2 (document → worker):** the `ImageBitmap` is sent to the ML worker with `postMessage(bitmap, [bitmap])`, a true zero-copy transfer. Image data never touches networking code.
6. **Coordinate frames:** every box records which frame it is in, and the fusion layer converts between them with one tested utility.

Frame	Origin	Units	Used by
Viewport	Top-left of visible area	CSS px	<code>getBoundingClientRect</code> , overlay
Page	Top-left of document	CSS px	Screen State bboxes (viewport + scroll offset)
Image	Top-left of screenshot	Device px (CSS px × <code>devicePixelRatio</code>)	Detector, OCR, face model
Crop	Top-left of a crop	Device px, then rescaled to model input	Tier 2 crops, model inputs
Frame-local	Top-left of an iframe	CSS px	Iframe content scripts; converted using the frame's offset chain

Browser zoom, `devicePixelRatio` and scroll are read at capture time. If the page scrolls between DOM snapshot and screenshot, the step is re-captured rather than corrected.

7. Frames are discarded immediately after perception; nothing is cached across steps except element embeddings for diffing.

4.3 DOM and accessibility extraction

The DOM is the primary, cheapest signal. A single walk produces a candidate element list.

- **Traversal:** `TreeWalker` over the document, including open shadow roots and same-origin iframes. Cross-origin iframes need host permission for their origin (Section 4.1); with it, a separate content-script instance reports up with frame offsets. Without it, the frame is an `opaque_frame` element with a bbox and no content.

- **Candidate filter:** interactive elements (`a` , `button` , `input` , `select` , `textarea` , `[role]` , `[contenteditable]` , `[tabindex]` , elements styled `cursor: pointer`) plus text blocks needed for context (headings, labels, error messages).
- **Per element:** tag, ARIA role (explicit or implicit), accessible name (computed from `aria-label` , `aria-labelledby` , `<label for>` , `placeholder` , `title` , `alt` , text content), input `type` and `autocomplete` , state (disabled, checked, expanded, selected, required, invalid), bbox from `getBoundingClientRect` , and parent landmark (form, nav, dialog).
- **Visibility:** drop elements with zero size, `display:none` , `visibility:hidden` , `opacity:0` , or fully clipped. Occlusion check: `document.elementFromPoint` at the bbox centre and 4 inner points; if another element covers it, mark `occluded: true` .
- **Element registry (two-way):** a `WeakMap<Element, id>` finds the ID of a node, and a `Map<id, WeakRef<Element>>` finds the node from an ID. A `FinalizationRegistry` plus a sweep on each step removes dead entries. IDs `e<n>` are never written into the page.
- **Document identity:** each document gets a `doc_id` that changes on navigation or major re-render (e.g. SPA route change). Element IDs are only valid with their `doc_id` .
- **Raw values stay local:** `input.value` , `data-*` attributes and hidden inputs are read only into the worker for PII classification and vault filling. They are never serialized raw.
- **Change tracking:** a `MutationObserver` marks dirty subtrees so step 2+ re-extracts only what changed.
- **Settle detection:** a step is "settled" when the DOM has had no relevant mutations for 300 ms (capped at 3 s) **and** the action's expected state is checkable. Network activity is only a weak extra signal: resource-timing entries appear when requests finish, so they cannot give a live count of pending requests. Where needed, a page-world script wraps `fetch` /XHR to count in-flight requests, with the known limits that it misses workers, WebSockets and requests started before injection.

Target: full extraction ≤ 40 ms for 2,000 DOM nodes; incremental ≤ 10 ms. The walk runs in `requestIdleCallback` chunks of ≤ 8 ms to avoid janking the page.

5. Client: local perception models

The client runs a small portfolio of specialised models (total ≤ 40 MB int8) instead of one large model, and each runs only when its tier is triggered. Sizes below are approximate and must be confirmed during the model spike (Milestone M1).

5.1 Model portfolio

Model role	Candidate	Approx. size (int8)	Input	Runs when	Target latency (WebGPU / WASM)
UI element detector	YOLO-nano class detector (e.g. YOLOv8n/YOLO11n) fine-tuned on web UI screenshots	3–6 MB	640×640 viewport	Tier 1 page present, first step of new page	25 ms / 120 ms
Text detection + recognition (OCR)	PaddleOCR mobile det + rec exported to ONNX	8–12 MB	Region crops only	Tier 1 region with no DOM text	15 ms per crop / 60 ms
Face detector	BlazeFace-class short-range detector	< 1 MB	128×128 tiles over image regions	Any <code>img</code> , <code>video</code> , <code>canvas</code> region is visible	5 ms / 15 ms
Icon classifier	MobileNetV3-small fine-tuned on ~150 common UI icons	2–3 MB	64×64 icon crops	Icon-only buttons with no accessible name	2 ms per icon / 8 ms
Text PII NER	Distilled small transformer token classifier (e.g. BERT-mini or DeBERTa-small class) fine-tuned on PII entities	10–18 MB	Text spans ≤ 128 tokens	Free text not caught by rules (names, addresses)	8 ms per batch / 40 ms

Two small additions support the crop allowlist (Section 6.8): a QR/barcode locator (native `BarcodeDetector` or a ~1 MB WASM detector) and a script-identification classifier (~1 MB) on OCR line crops. Both run only on crops about to be transmitted, so they add nothing to Tier 0 and Tier 1 steps.

Rejected for v1: running a general small VLM (Florence-2, SmolVLM, Moondream) on every frame. They are 200M+ parameters, 150–500 MB, and cost hundreds of milliseconds per image on consumer GPUs. They stay a v2 experiment for Tier 1 on high-end devices.

5.2 Runtime

- **Library:** ONNX Runtime Web (`onnxruntime-web`) as the single inference runtime. Transformers.js is acceptable for the NER model because it wraps the same runtime and handles tokenization.

- **Backend selection at startup:** try `webgpu` → fall back to `wasm` with SIMD + threads (needs cross-origin isolation in the offscreen document) → fall back to single-thread `wasm`. Record the chosen backend in telemetry.
- **Loading:** models are downloaded once from our CDN, verified by SHA-256, stored in IndexedDB, and lazily instantiated. Detector and face model warm up on extension start; OCR and NER load on first need.
- **Threading:** all inference runs inside a dedicated Web Worker. The content script and page main thread never run ML.
- **Batching:** OCR crops and icon crops are batched per step; NER spans are batched up to 16 per call.
- **Memory:** tensors are explicitly released after each step; a single session per model is reused.

5.3 Optimisation

1. **Quantization:** int8 dynamic quantization for NER and classifier; int8 static (calibrated on 500 UI screenshots) for the detector; keep fp16 on WebGPU where int8 kernels are slow and measure both.
2. **Resolution discipline:** detector sees the viewport at 640 px long side; OCR and icon models see only crops. Never run a model on the full-resolution screenshot.
3. **Skip logic for perception only:** if the DOM covers a region with confident text and role, the detector does not need to find that element for planning. This never applies to privacy: any pixels that will be transmitted go through the full pixel privacy pipeline (Section 6.4).
4. **Diffing:** on step 2+, only regions whose DOM subtree mutated or whose pixels changed (perceptual hash per 64×64 tile) are re-processed.
5. **Weak devices keep privacy, lose speed or features.** The NER model is never silently dropped. On WASM-only or < 4 GB RAM devices the client first switches to a smaller validated NER model (e.g. a ~4 MB tiny token classifier) that must meet the same recall target on the PII benchmark. If no validated detector can run, the client enters **restricted egress mode** (Section 6.7): free text that has not been checked by a detector is not sent. Only reduced-accuracy perception is acceptable on weak devices; reduced privacy is not.

5.4 Training data for local models

Model	Data sources	Labels needed
UI detector	Public web UI datasets (e.g. WebUI, Rico for mobile transfer), plus auto-labelled screenshots from our own crawl where DOM bboxes provide free labels	Classes: button, input, checkbox, radio, dropdown, link, icon, image, text block, toggle, tab, plus privacy classes avatar, photo, video, document_image, handwriting, map, chart
Icon classifier	Icon font sets (Material, Font Awesome, Tabler) rendered at many sizes and colours	Icon meaning label
PII NER	Synthetic forms and pages generated with Faker (Indian locale included: Aadhaar, PAN, IFSC, +91 phones), plus public PII NER corpora	BIO tags per PII class
Face detector	Use pretrained weights; validate on screenshots of video-call and social UIs	None beyond validation set

The DOM-as-free-labels trick is the key data strategy: render thousands of pages headlessly, take DOM bboxes and roles as ground truth, and train the detector to find the same elements from pixels. It then generalises to canvas and custom widgets where the DOM is missing.

DOM-derived labels only cover elements the DOM exposes, so they do not prove the detector works on canvas apps. Canvas and custom-widget pages are hand-labelled, and those pages are reported as their own test slice.

Data splits are by site family, not by page. Training, validation and test sets are split by website or template family (e.g. all pages of one portal, or all sites built on one form builder, go in one split). Swapping names and numbers within the same template is a weak generalisation test and is not used for reported numbers.

6. Client: PII detection and redaction engine

PII is detected by layering four independent detectors (semantic DOM, pattern rules, NER, vision) and redacting on the union, because missing PII is far costlier than overmasking. Structure is always preserved: the server learns that a field is an email, never what the email is.

6.1 Fusion first: the element graph

Fusion builds two different outputs, and they must not be confused.

- **Interaction map (for planning):** DOM elements and visual detections are merged into one list. A visual detection matches a DOM element when $IoU \geq 0.5$; matched pairs keep the DOM text as the element's name. Unmatched visual detections become `src: "vision"` elements with OCR text.
- **Privacy evidence (for redaction):** every text source is kept separately, even for matched pairs: DOM text, DOM attribute values, and OCR text of the rendered pixels. A matched box does not mean the rendered pixels say the same thing as the DOM (an image button labelled "Profile" may show a person's name). Each source is scanned independently, and the redaction findings are merged by union.

The low-confidence rules also differ. Low confidence about **what an element does** may justify Tier 2 escalation. Low confidence about **whether a region contains personal data** means the region is masked or withheld, never escalated.

6.2 Detector layers

Layer	Signal	Catches	Cost
L1 Semantic DOM	<code>type=password</code> , <code>autocomplete</code> tokens (<code>cc-number</code> , <code>email</code> , <code>tel</code> , <code>name</code> , <code>street-address</code> , <code>bday</code> , <code>one-time-code</code>), label/name keywords ("Aadhaar", "PAN", "account no", "CVV")	Form fields by purpose, even when empty	~0 ms

Layer	Signal	Catches	Cost
L2 Pattern rules	Regex + checksums: email, phone (+91 and E.164), card number (Luhn), Aadhaar (12 digits, Verhoeff), PAN ([A-Z]{5}[0-9]{4}[A-Z]), IFSC, IBAN, UPI IDs, IPs, dates of birth, OTPs near "OTP" labels, JWT/API-key shapes	Values in any text or input	< 2 ms
L3 Text NER	Local NER model (Section 5.1)	Person names, addresses, organisations tied to a person, free-text PII	~8 ms per batch
L4 Vision	Face detector; OCR text re-fed to L2/L3; image regions of type avatar/profile photo	Faces, text inside images/canvas, ID-card photos	Tier 1 only
L5 Policy	Per-site and per-user rules: always-redact selectors, allowlisted domains, user-marked sensitive regions	Site-specific leaks (e.g. account balance widget)	~0 ms

Decision rule: an item is sensitive if **any** high-precision layer (L1, L2 with checksum, L5) fires, or if L3/L4 score ≥ 0.5 . Thresholds are tuned per class on the validation set to hit $\geq 98\%$ recall, then precision is maximised.

6.3 PII classes and default treatment

Class	Risk	Text treatment	Pixel treatment (Tier 2 crops)
Password, OTP, CVV, PIN	Critical	Value dropped entirely: <code>{{SECRET}}</code> with <code>filled: true/false</code>	Solid black box
Card number, bank account, Aadhaar, PAN	Critical	Typed placeholder <code>{{CARD_1}}</code> + format hint <code>last4_present: false</code>	Solid black box

Class	Risk	Text treatment	Pixel treatment (Tier 2 crops)
Email, phone, UPI ID	High	Typed placeholder {{EMAIL_1}}	Solid box with class label
Person name, address, DOB	High	Typed placeholder {{NAME_1}}, {{ADDR_1}}	Solid box with class label
Face, profile photo	High	Element marked image: face_redacted	Opaque fill of the whole face or avatar container, labelled "face". Blur and pixelation are not used: people can still be identified from blurred or pixelated faces
Free text with embedded PII	Medium	Span-level replacement inside the text	Span boxes blacked
Balances, amounts, order IDs	Medium (policy)	{{AMOUNT_1}} when L5 policy enables it	Solid box

Placeholders are consistent within a session: the same email always maps to {{EMAIL_1}}, so the server can reason that two fields hold the same value without seeing it.

6.4 Redaction operations

1. **Text redaction:** replace detected spans in element text, labels and values with placeholders. Labels that merely describe a field ("Email address") are kept; only values are replaced.
2. **Value omission:** input values are never sent unless non-sensitive and short (e.g. a selected country). Default is value_state: "empty" | "filled" | "redacted".

- 3. **Pixel redaction, full pipeline on every crop:** any crop that will be transmitted is checked in full, whatever the reason for sending it and even if the DOM was enough for planning. That means face detection on the whole crop, OCR on the whole crop, then pattern rules and NER on the OCR text, then masking of every flagged box and of every region outside the supported content classes in Section 6.8. Masks are solid fills, dilated by 4 px to cover anti-aliasing and partial glyphs. The crop is re-encoded as JPEG, which also strips metadata.
- 4. **Semantic obfuscation:** for context text (e.g. "Welcome back, Priya"), replace with "Welcome back, {{NAME_1}}" rather than deleting, so meaning survives.
- 5. **Verification pass:** after redaction, pattern rules re-run over the serialized payload and OCR plus NER re-run over each redacted crop. Any hit blocks the send (Section 10.2). If a crop cannot be fully checked (e.g. OCR model unavailable), it is not sent.

6.5 The vault

- A map of placeholder → {value, class, source} held only in the ML worker’s memory.
- Scoped to one task session; cleared on task end, tab close, or 15 min idle.
- Also stores user-provided task values: if the user types "change my phone to 98xxxxxx12" in the side panel, that number is vaulted as {{PHONE_NEW}} before the task text is sent to the server.
- Never written to IndexedDB, logs or telemetry. Crash reports exclude worker memory.

Every vault entry carries a use binding, enforced locally. The server can propose using a placeholder, but only the client decides whether it may be released.

Binding field	Meaning	Example
task_id	The task the value belongs to	Only the "update phone" task
origin	Site allowed to receive it	https://portal.example.in
doc_id	Document it was approved for	The current profile-edit page
allowed_targets	Destination fields, matched by role + name + autocomplete / type + form	The "Mobile number" tel input in the Profile form
operations	What may be done with it	type only; never navigate , extract to server, or say

If an action puts a placeholder into any other field (a search box, a chat composer, a different form, another origin), the executor refuses locally and reports `binding_denied`. The server's `risk` field never overrides this check. For values the user typed in the task, the destination is approved when the planner first proposes it, with a one-tap confirmation that names the field and site.

Where the privacy boundary actually is. The vault protects values from the AI server. It cannot protect them from the website: once a value is typed into an input, it stays in the input's `value`, and page scripts can read, copy or send it at any time, just as when the user types it. Giving a value to a website is therefore an authorisation decision, which is why destinations are bound and confirmed.

6.6 Text-to-pixel alignment

Precise redaction needs every detected text span mapped back to the exact pixels or DOM range it came from.

1. **Normalize with an offset map:** text is Unicode-normalized (NFC), whitespace collapsed and case-folded for matching, while keeping a map from every normalized index back to the original string index.
2. **Tokens to characters:** the NER tokenizer's offset mapping converts token-level tags to character spans in the normalized text, then through the offset map to original spans.
3. **DOM text to rectangles:** original spans are turned into a DOM `Range` over the text nodes; `Range.getClientRects()` returns one rectangle per line, which handles wrapped text.
4. **OCR text to boxes:** the OCR model returns word or character boxes; spans are mapped to the boxes they overlap. A partial-word match masks the whole word.
5. **Hindi and other complex scripts:** Devanagari conjuncts and matras mean one visible glyph cluster can cover several code points. Alignment works on grapheme clusters (`Intl.Segmenter`), and masks cover whole clusters.
6. **Uncertain alignment:** if a span cannot be mapped confidently (OCR text differs, overlapping lines), the whole line or text block is masked.

6.7 Restricted egress mode

The privacy contract is defined by what may be sent, not by how good the detectors are. When a required detector is unavailable or fails, content falls back to the next-safer row.

Content	All detectors available	NER unavailable	OCR or face model unavailable
Structural fields (role, element type, bbox, state)	Sent	Sent	Sent
Short UI labels (button, link, field labels ≤ 6 words, matched against a UI-vocabulary allowlist)	Sent after rules + NER	Sent if on the allowlist, else masked	Sent after rules + NER
Free text (paragraphs, messages, names in headings)	Sent after rules + NER	Not sent; replaced by <code>{{TEXT_WITHHELD}}</code> with length and landmark	Sent after rules + NER
Image crops (Tier 2)	Sent after full pixel pipeline	Not sent	Not sent

If the task cannot proceed without withheld content, the planner gets `ask_user` or switches to Guide mode, so the user acts locally. The side panel shows when restricted mode is on and why.

6.8 Supported content classes (crop allowlist)

A crop may leave the device only if every region in it belongs to a supported content class that the pipeline can check. Running face detection, OCR and NER proves those checks ran; it does not prove the crop holds nothing else sensitive. So Tier 2 crops use an allowlist, not a blocklist: anything the pipeline cannot positively classify and check is masked, and if masking would leave nothing useful, the crop is withheld.

Content class	Detected by	Checked by	Disposition in a transmitted crop
UI chrome (buttons, inputs, borders, backgrounds)	UI detector	Structure only; carries no personal data by itself	Sent

Content class	Detected by	Checked by	Disposition in a transmitted crop
Icons from the known icon set	Icon classifier (conf $\geq$ 0.8)	Icon label	Sent
Printed text, supported scripts (Latin, Devanagari)	OCR + script identification	Pattern rules + NER on OCR text	Sent after span masking
Printed text, other scripts (e.g. Tamil, Bengali, Arabic, CJK)	Script identification (small classifier on OCR line crops)	No validated PII detector	Whole text block masked
Text OCR cannot read confidently (mean char conf < 0.7, stylised, tiny)	OCR confidence	Cannot be checked	Region masked
Faces, avatars, profile photos	Face detector + UI detector <code>avatar</code> class	—	Opaque fill (Section 6.3)
QR codes and barcodes	Barcode detector (see below)	Not decoded; content is never inspected	Always masked
Handwriting and signatures	UI detector <code>handwriting</code> class; <code>signature-pad</code> <code><canvas></code> elements from DOM	Cannot be checked	Always masked
Photos of documents or cards (ID, cheque, bill)	UI detector <code>document_image</code> class	Cannot be checked reliably	Whole image masked
Maps	UI detector <code>map</code> class; known map embeds from DOM	Location is sensitive	Masked unless the task is about the map, and then only after user approval per crop
Charts and graphs	UI detector <code>chart</code> class	Axis and label text via OCR + NER	Sent after span masking (a main Tier 2 use case)

Content class	Detected by	Checked by	Disposition in a transmitted crop
General photos and video frames	UI detector <code>photo</code> / <code>video</code> class	Faces masked; the rest cannot be checked	Masked by default; sent only with user approval per crop
Anything else, or detector conf < 0.5	—	—	Masked

Barcode detector: QR codes can encode contact cards, Wi-Fi passwords, payment IDs and login tokens with no readable text or face. The client adds a finder-pattern detector for QR and common 1D barcodes: the native `BarcodeDetector` API where available (Chromium on some platforms, not Firefox), else a small WASM detector (`zxing-class`, ~1 MB) run only on crops being transmitted. The detector only locates codes; the client never decodes them, since masking does not need the content.

Cost rule for new detectors: a specialised detector is added only when (a) the content class appears in real Tier 2 crops often enough to matter and (b) masking the class outright would hurt task success. Until then, the class is simply masked. QR/barcode detection passes this test because it is cheap and runs only on the ~3% of steps that escalate; handwriting and document images are handled as UI-detector classes rather than separate models.

Supported languages (v1): English and Hindi for NER; Latin and Devanagari for OCR and pattern rules. Text in other scripts is treated as unsupported in crops (masked) and in structured text (treated as free text without NER coverage, so restricted egress rules in Section 6.7 apply). Adding a language means adding it to the OCR, script-ID and NER benchmarks first.

Each transmitted crop carries a manifest listing every region, its class, and the checks applied. The Egress Guard rejects a crop whose manifest has any region not in the allowlist or not masked.

7. Wire protocol

Client and server speak a versioned JSON protocol with two core messages: the client's Screen State and the server's Action. Both are defined once in a shared JSON Schema package, which generates TypeScript types for the client, Pydantic models for the server, and the grammar used for constrained decoding.

7.1 Endpoints

Method	Path	Purpose
POST	<code>/v1/sessions</code>	Start a task: sanitized task text, allowed domains, client capabilities → <code>session_id</code>
POST	<code>/v1/sessions/{id}/step</code>	Send Screen State (full or diff) → one Action (or a short plan + next Action)
POST	<code>/v1/sessions/{id}/escalate</code>	Send redacted crops for Tier 2 visual questions → structured answer
POST	<code>/v1/sessions/{id}/feedback</code>	Execution result of the last action (success, error, page changed)
DELETE	<code>/v1/sessions/{id}</code>	End session; server purges all state

Transport: HTTPS with TLS 1.3, gzip bodies, optional streaming via Server-Sent Events for the Action so execution can start as soon as the JSON closes. Auth: per-install key pair; requests signed, short-lived tokens.

7.2 Screen State (client → server)

```
{
  "protocol": "1.0",
  "session_id": "s_8f2c",
  "step": 3,
  "kind": "diff",
  "page": {
    "url": "https://portal.example.in/profile/edit",
    "title": "Edit profile",
    "viewport": {"w": 1440, "h": 900, "scroll_y": 0, "page_h": 2100}
  },
  "redaction_scheme": "v1",
  "elements": [
    {"id": "e12", "role": "textbox", "name": "Full name", "input_type":
"text",
      "value_state": "redacted", "value": "{{NAME_1}}", "bbox": [420, 210,
360, 40],
      "state": {"required": true}, "landmark": "form:Profile", "src": "dom",
"conf": 1.0},
    {"id": "e14", "role": "textbox", "name": "Mobile number", "input_type":
```

```

"tel",
  "value_state": "redacted", "value": "{{PHONE_1}}", "pii_class": "phone",
  "bbox": [420, 290, 360, 40], "landmark": "form:Profile", "src": "dom",
"conf": 1.0},
  {"id": "e19", "role": "button", "name": "Save changes", "bbox": [420,
520, 160, 44],
  "state": {"disabled": false}, "src": "dom", "conf": 1.0},
  {"id": "v3", "role": "button", "name": "settings (icon)", "bbox": [1380,
20, 32, 32],
  "src": "vision", "conf": 0.81},
  {"id": "v7", "role": "img", "name": "profile photo", "image":
"face_redacted",
  "bbox": [120, 200, 160, 160], "src": "vision", "conf": 0.97}
],
"removed": ["e9"],
"text_context": ["Welcome back, {{NAME_1}}", "Your number is verified"],
"last_action_result": {"action_id": "a2", "status": "ok", "page_changed":
true}
}

```

Rules:

- **id** prefixes: **e** = DOM-backed, **v** = vision-only. Vision-only elements are executed by coordinates.
- Every Screen State carries **observation_id** (monotonic per session), **doc_id** (Section 4.3) and **seq** (request sequence number). Element IDs are only meaningful together with their **doc_id**.
- Elements are ordered top-to-bottom, left-to-right. Off-screen elements are included with **bbox** so the planner can request scrolling, capped at 300 elements; beyond that, only the viewport plus landmarks are sent.
- **pii_class** is present when a value was redacted, so the planner knows the field's nature.
- No free-form HTML, no attributes, no CSS, no raw text outside **name**, **value** and **text_context**.
- Typical size: 2–6 KB gzipped for a form page, roughly 1,500–4,000 tokens.

7.3 Action (server → client)

```
{
  "action_id": "a3",
  "observation_id": 7,
  "doc_id": "d_41",
  "thought": "Mobile field holds the old number; replace it with the new one from the task.",
  "action": {"type": "type", "target": "e14", "text": "{{PHONE_NEW}}"},
  "clear_first": true,
  "expect": {"element": "e14", "value_equals": "{{PHONE_NEW}}"},
  "consequential": false,
  "risk": "low",
  "progress": {"step": 3, "subgoal": "update phone", "remaining_estimate": 2}
}
```

The client checks `value_equals` locally by comparing the field's final value with the vault value, after the same normalization the site applies (e.g. stripping spaces). Only the boolean result goes back to the server. `consequential: true` marks actions that change state outside the page (submit, send, pay, delete); the client also infers this independently from the target (Section 9.4) and uses whichever is stricter.

Action type	Required fields	Notes
<code>click</code>	<code>target</code>	DOM click; for <code>v*</code> targets, pointer events at bbox centre
<code>type</code>	<code>target</code> , <code>text</code> , <code>clear_first</code>	<code>text</code> may contain placeholders; resolved locally
<code>select</code>	<code>target</code> , <code>option</code>	Option by visible label
<code>scroll</code>	<code>direction</code> , <code>amount</code> or <code>to_element</code>	
<code>press_key</code>	<code>key</code>	Enter, Tab, Escape only in v1
<code>wait</code>	<code>ms</code> or <code>until</code>	<code>until</code> : element appears/disappears
<code>navigate</code>	<code>url</code>	Only within session's allowed domains
<code>extract</code>	<code>target</code> , <code>field</code>	Returns structured data to the user locally; the server never sees the result

Action type	Required fields	Notes
ask_user	question , options?	Pauses loop; shown in side panel
escalate	targets , question	Requests Tier 2 crops of named elements
done	summary	Task complete; summary may contain placeholders, resolved locally for display

7.4 Redaction-aware contract

The server is explicitly told how to treat redacted content, via the system prompt and schema:

- 1. Placeholders are opaque, typed references. The planner may copy them into type actions but must never invent values or placeholders that do not exist in the state or task.
- 2. value_state: "redacted" means "a value exists"; "empty" means the field needs filling.
- 3. image: face_redacted , {{TEXT_WITHHELD}} and masked crop regions are intentional. The planner must not ask for the unredacted version.
- 4. The server validates every returned action: target must exist in the current state, navigate URLs must match the allowlist. Invalid actions are re-generated once, then turned into ask_user .
- 5. **Server validation is not authorisation.** The client independently enforces placeholder bindings (Section 6.5), doc_id / observation_id freshness and consequential-action confirmation. An action that passes server checks can still be refused locally.

7.5 Protocol extensions for the assistant modes

The protocol gains a mode field, guide-mode actions, voice fields and multi-tab context. All additions are sanitized by the same rules as Section 7.2.

Session start additions: mode: "ask" | "guide" | "agent" | "background" , voice: true|false , granted_tabs: [tab ids] , read_only: true|false .

Screen State additions:

```

{
  "mode": "guide",
  "user_turn": {"source": "voice", "text": "how do I download my fee
receipt", "interrupt": false},
  "user_interaction": {"type": "click", "target": "e31", "after_step": "g2"},
  "tabs": [
    {"tab": "t2", "title": "Laptop A – {{STORE_1}}", "digest": "16 GB RAM,
512 GB SSD, price {{AMOUNT_1}}..."},
    {"tab": "t3", "title": "Laptop B", "digest": "8 GB RAM ..."}
  ]
}

```

- `user_turn` carries the sanitized transcript or typed message; `interrupt: true` marks a barge-in during an agent task.
- `user_interaction` tells the planner what the user did in Guide mode, so it can advance or correct.
- `tabs` is present only in Ask mode when the user references other tabs. Amounts are placeholders only if the L5 policy redacts them (Section 6.3); in a price comparison the user can allow amounts per task.

New server outputs:

Output	Mode	Fields	Client behaviour
<code>answer</code>	Ask	<code>text</code> , <code>citations: [{tab, element?}]</code>	Render in panel, resolve placeholders locally, citations focus tab and highlight
<code>point</code>	Guide	<code>target</code> , <code>say</code> , <code>style</code> (pointer, circle, spotlight), <code>advance_on</code> (click, input, navigate)	Draw overlay, speak <code>say</code> , wait for the interaction
<code>annotate</code>	Guide	<code>shapes: [{kind, target, label}]</code> , <code>say</code>	Draw several marks at once, e.g. "these three fields are required"
<code>say</code>	All	<code>text</code>	Speak and show without other action

Output	Mode	Fields	Client behaviour
<code>switch_tab</code>	Agent, Background	<code>tab</code>	Focus or operate in a granted tab
<code>report</code>	Background	<code>summary</code> , <code>items</code>	Notification + stored result, placeholders resolved locally

Every `point` and `annotate` target must exist in the current state, validated exactly like action targets. The `say` field is limited to 25 words to keep TTS short.

7.6 Sequencing, diffs and resynchronisation

Diff-based Screen States save tokens but can drift out of sync, so the protocol defines what happens when messages are lost, repeated or late.

Situation	Rule
Diff references a base the server does not have	Server replies <code>resync_required</code> ; client sends a full state
Every Nth step (default 5), after navigation, or on <code>doc_id</code> change	Client sends a full state regardless
Duplicate request (same <code>seq</code> , e.g. network retry)	Server returns the cached response for that <code>seq</code> ; it never plans twice
Action refers to an old <code>observation_id</code> or other <code>doc_id</code>	Client discards it without executing and re-observes
Response arrives after the user pressed stop or changed the task	Client drops it; responses carry <code>seq</code> and task version
Target element changed since observation (different role, name or bbox beyond tolerance)	Client re-observes and asks for a new action; it never guesses a substitute
Client-side state hash differs from server's view	Server includes a hash of the state it planned against; mismatch triggers resync

8. Server: planning and escalation

The server runs a small open-weight text LLM as the default planner with grammar-constrained JSON output, and a small open-weight VLM only for Tier 2 escalations. Both are self-hostable; cloud-hosted instances of the same weights are allowed for demos.

8.1 Model choices

Role	Primary candidate	Fallback	Why
Planner (text)	Qwen2.5-7B-Instruct class or its current successor	Llama 3.1 8B Instruct class; a 3B model for low-cost mode	Strong tool-use and JSON adherence at 7–8B; fits one 24 GB GPU at fp16 or int8
Escalation VLM	Qwen2.5-VL-3B/7B class	Florence-2 for OCR-style questions only	Trained on documents and UIs; small crops keep image tokens low
Upgrade path	Fine-tuned planner on our trajectory data (Section 12)		Biggest accuracy lever after v1

Model versions move quickly; confirm the latest open-weight releases and licences during M1. Licences must permit commercial self-hosting.

8.2 Serving stack

- **Inference server:** vLLM (OpenAI-compatible API) with prefix caching and continuous batching. SGLang is the alternative if its structured-output speed wins in benchmarks.
- **Constrained decoding:** vLLM guided decoding (xgrammar or Outlines backend) using the Action JSON Schema, so every response is schema-valid by construction. This removes format hallucination; it does not remove wrong-choice errors, which validation (Section 7.4) and verification (Section 9) handle.
- **Prompt caching:** the system prompt, tool schema and redaction rules form a fixed prefix (~1,200 tokens) that stays in the KV cache across all sessions.
- **API layer:** FastAPI + Pydantic, Redis for session state (TTL 30 min), no persistence of Screen States beyond the session unless the user opts in to share anonymised traces.
- **Hardware target:** one NVIDIA L4 or A10G (24 GB) serves the 7B planner at int8 for roughly 10–20 concurrent sessions; the VLM shares or gets a second GPU.

8.3 Planner prompt structure

1. **System (cached):** role, action schema, redaction contract, safety rules (never submit payments without `ask_user` , never navigate off allowlist).
2. **Task:** sanitized user goal, e.g. "Change my mobile number to {{PHONE_NEW}} on this portal."
3. **Memory:** compressed history of the last 5 steps: action taken + result, not full past states.
4. **Current Screen State:** the JSON from Section 7.2, re-rendered to a compact line format to save tokens, e.g. `[e14] textbox "Mobile number" tel =REDACTED: {{PHONE_1}} @420,290 .`
5. **Instruction:** "Return exactly one next action."

The `thought` field is limited to 40 tokens. Short reasoning improves choices; long reasoning costs latency.

8.4 Latency levers on the server

Lever	Effect
Compact line format for elements	~40% fewer input tokens than raw JSON
Prefix cache hit on system prompt	Prefill cost only for task + state
Diff states after step 1	Planner sees only changes plus a short persistent summary
Output capped at ~80 tokens	Decode time is the dominant cost; keep it short
Streaming action	Client starts resolving the target before <code>risk / progress</code> arrive
Speculative multi-action plans	For predictable form fills, return up to 3 actions with per-action <code>expect</code> ; client executes in sequence and stops on first mismatch

8.5 Escalation (Tier 2)

- Triggered by the client (local conf < 0.6 on a needed element) or by the planner via an `escalate` action.

- Payload: up to 2 redacted crops, each ≤ 512 px long side, plus the element IDs they cover and a narrow question ("What does this icon do?", "Which bar is the highest?").
- VLM output is also schema-constrained: `{element_id, role, label, state, answer}`. Results are merged into the planner's state as if they came from the client, tagged `src: "server_vlm"`.
- Crops are processed in memory and discarded; they are never logged.

9. Action executor and agent loop

The executor turns each validated Action into real browser events, verifies the expected outcome, and reports back; risky actions always pause for the user.

9.1 Agent loop states

```
stateDiagram-v2
    [*] --> Observe
    Observe --> Plan: Screen State sent
    Plan --> Confirm: risk = high
    Plan --> Execute: risk = low/med
    Confirm --> Execute: user approves
    Confirm --> Observe: user edits/declines
    Execute --> Verify
    Verify --> Observe: expectation met
    Verify --> Recover: mismatch or error
    Recover --> Observe: retry (max 2)
    Recover --> AskUser: retries exhausted
    Plan --> Done: action = done
    Plan --> AskUser: action = ask_user
    AskUser --> Observe
    Done --> [*]
```

The loop stops after 30 steps or 5 minutes per task, whichever comes first, and hands control back to the user.

9.2 Execution

1. **Check freshness:** the action's `doc_id` and `observation_id` must match the current document and the latest observation; otherwise discard and re-observe (Section 7.6).

2. **Resolve target:** look up the element through the two-way registry (Section 4.3). If the node is gone or its role, name or bbox changed beyond tolerance, do **not** substitute a similar control; re-observe and ask the planner again. Repeated controls (e.g. many "Remove" buttons) make guessing unsafe.
3. **Authorise placeholders:** check each placeholder's binding (Section 6.5) against this target, origin and operation, then fetch the value from the worker. Once typed, the value lives in the input like any user-typed value.
4. **Scroll into view** (`scrollIntoView({block: "center"})`) and re-check occlusion.
5. **Perform the action through a control adapter.** Events created with `dispatchEvent` are synthetic (`isTrusted === false`), and some sites or default behaviours ignore them. So the executor uses the most reliable method per control type, and each adapter is tested:

Control	Method	Known limits
Button, link, checkbox, radio	<code>element.click()</code> (runs default activation) after focus	Sites that check <code>isTrusted</code> reject it
Text input, textarea	Focus, native value setter, then <code>input</code> and <code>change</code> events (works with React/Vue controlled inputs)	Synthetic key events do not insert text, so per-key simulation is not used for typing
Contenteditable / rich editors	<code>beforeinput</code> + <code>execCommand('insertText')</code> where supported, else editor-specific adapter	Editor-dependent; unsupported editors go to Guide mode
Native <code><select></code>	Set <code>value</code> / <code>selectedIndex</code> , dispatch <code>change</code>	Custom dropdowns use the click adapter per option
Keyboard (Enter, Tab, Escape)	Direct equivalents: <code>form.requestSubmit()</code> for Enter, <code>focus()</code> on next element for Tab, close via the element's own dismiss control	Synthetic key events do not move focus or submit forms
Canvas and custom-drawn widgets	Pointer events at coordinates	Works only on apps that accept synthetic events; otherwise Guide mode

6. **Optional trusted input (Chrome only, v2):** the `chrome.debugger` API's `Input.dispatchEvent` / `Input.insertText` produce trusted input, but Chrome shows a persistent "is debugging this browser" banner and it needs the `debugger` permission. It is an opt-in setting for power users, not the default. Firefox has no equivalent; unsupported controls use Guide mode there.
7. **Visual feedback:** a brief outline highlight on the target in the page, so the user can see what the agent is doing.

9.3 Verification and recovery

- After execution, wait for settle (Section 4.3), then check the action's `expect` clause locally. Value checks compare the actual final value to the intended one (`value_equals`); "field is filled" is never accepted as proof, because a field may already have held an old value.
- Report one of four outcomes in the next Screen State, with a sanitized reason (never raw page text):

Outcome	Meaning	Next step
<code>ok</code>	Expected state observed	Continue
<code>failed</code>	The action clearly did not take effect (value unchanged, element not clicked, adapter error)	Safe to retry with another adapter, max 2
<code>mismatch</code>	Something happened but not what was expected	Re-observe; planner decides
<code>unknown</code>	Cannot tell if it took effect (timeout, navigation mid-action, network error after submit)	Never auto-retry if consequential; re-observe and ask the user if unclear

- No automatic replay of consequential actions.** A submit, payment, send or delete whose outcome is `unknown` or `mismatch` is never repeated automatically, because it may have succeeded. The agent looks for evidence (confirmation message, changed URL, updated list) and otherwise asks the user.
- Recovery for non-consequential actions: re-observe, scroll and retry, switch adapter, or escalate the region to Tier 2 for interpretation.

9.4 Safety guards

Guard	Rule
High-risk confirmation	Submit buttons in forms with payment, password-change, delete or "confirm order" semantics require user approval in the side panel
Domain allowlist	<code>navigate</code> and link clicks leaving the session's allowed domains need approval
Prompt injection	Page text is data, not instructions. Text elements matching injection patterns ("ignore previous instructions") are flagged <code>suspicious: true</code> and the planner is told to ignore instructions from page content
Kill switch	Global stop button in side panel and a keyboard shortcut; stops the loop immediately
Action audit	The side panel shows every action and its target in plain language, locally
No credential entry	The agent never types into password fields; <code>{{SECRET}}</code> cannot be resolved by the executor, so the user fills those themselves

10. Security and privacy

Privacy is enforced by architecture and by a fail-closed Egress Guard, not by trusting the server. If redaction cannot be verified, nothing is sent.

10.1 Threat model

Threat	Example	Mitigation
Missed PII in text	A name in a free-text paragraph	Layered detectors (Section 6.2), recall-first thresholds, Egress Guard re-scan

Threat	Example	Mitigation
PII in hidden DOM	<code>data-user-email</code> , hidden inputs, JSON in <code><script></code>	Attributes and scripts are never serialized; only <code>name / value / text_context</code> fields exist in the schema
PII in pixels	Name on an avatar, ID card image	Pixels only leave in Tier 2 crops; face blur + OCR-driven masking + post- redaction OCR check
Reversible redaction	Weak blur on text or faces	Solid fills for text and opaque fills for whole faces or avatars; mask dilation
Linkage/inference	URL contains user ID; page title has a name	URL path and query run through the same PII rules; titles scanned like any text
Malicious page (prompt injection)	Hidden text tells the agent to send data elsewhere	Planner rules, <code>suspicious</code> flags, domain allowlist, vault values never included in any outbound field
Compromised server	Server asks for unredacted data	Protocol has no field to carry raw data; client ignores such requests
Extension supply chain	Tampered model weights	SHA-256 pinned hashes, signed releases, minimal permissions
Logs and telemetry	Debug logs capture payloads	Telemetry is metrics-only (latencies, counts, backend); no payload logging in production builds

10.2 Egress Guard

Every outbound request passes through one module in the background context; no other code holds a network client.

1. **Schema check:** payload must validate against the protocol schema; unknown fields are rejected.
2. **Detector-coverage check:** every text field must carry a record of which detectors scanned it. Free text without NER coverage, or a crop without a full pixel-pipeline record, is rejected (Section 6.7).

3. **Vault check, field-aware:** each vault value and its normalized forms (no spaces, lowercase) is searched only in text-bearing fields (`name` , `value` , `text_context` , digests, transcripts), not in coordinates, IDs or sequence numbers. Partial forms such as a card's last 4 digits are matched only as whole tokens next to card-like context ("ending in", "xxxx"), so unrelated numbers do not cause false blocks.
4. **Rule re-scan:** pattern rules run over all text-bearing fields.
5. **Image check:** each crop must carry a manifest in which every region has a supported content class and its required checks, or is masked (Section 6.8); OCR plus NER re-run on the crop must return no hits, and the barcode locator must find no unmasked code.
6. **Decision:** pass, or block and surface "Blocked a possible leak" in the side panel with an option to mark the region sensitive and retry.

The Egress Guard is a last line of defence, not the main detector. It can only catch what it can recognise: known vault values and patterns. A name that no earlier stage detected will not match either, which is why coverage (step 2) and restricted egress mode matter more than this final scan.

10.3 User-facing transparency

- A **Privacy view** in the side panel shows exactly what the last request contained, rendered as the server would see it, including redacted crops.
- Users can add sites to "never send" and mark regions as always sensitive (feeds L5 policy).

10.4 Leak testing

- **Canary tests:** synthetic pages seeded with unique fake PII (canary strings and synthetic faces). An intercepting proxy in CI captures all outbound traffic, **decompresses** bodies (gzip, SSE streams), decodes base64 and URL encoding, then searches for canaries. Captured images are run through OCR and face detection to catch canaries that leaked as pixels.
- **Adversarial suite:** PII split across elements, rendered in canvas, inside SVG text, in image alt text, in URL params, in shadow DOM, in image buttons with generic labels, in Hindi/Devanagari, in QR codes and barcodes (contact cards, UPI payment codes, Wi-Fi codes), in handwritten signatures and signature pads, in photos of ID cards, and in unsupported scripts such as Tamil or Bengali.
- **Degraded-device suite:** the same canary pages run with NER, OCR and face models disabled one at a time, to prove restricted egress mode holds.

- **Compliance mapping:** design supports data-minimisation duties under India's DPDP Act 2023 and GDPR; legal review before launch.

10.5 Privacy and safety for voice, tabs, background tasks and memory

Comet's published history shows the main risk of an agentic browser: independent teams (Brave, LayerX, Trail of Bits, Zenity and others) demonstrated indirect prompt injection that made the agent exfiltrate data or act on phishing pages ([summary](#)). The same review notes Comet processes page content on its servers. Our design treats both as requirements.

Surface	Risk	Control
Voice	Raw audio contains voice biometrics and spoken PII	Audio never leaves the device; local STT; transcripts redacted; audio buffers discarded after transcription
Mic access	Always-listening fear	Push-to-talk by default; no wake word in v1; visible listening state on companion and toolbar icon
Screen perception	Constant observation	Perception runs only on hotkey or during an active task, the same promise HeyClicky makes; idle = no capture
Multi-tab	Reading tabs the user did not intend (e.g. a bank tab)	Explicit tab grant per task; sites on the "never send" list are excluded from digests; digests redacted locally
Cross-tab injection	A malicious tab instructs the agent to act on another tab	Content from one tab can never authorise an action in another; <code>switch_tab</code> requires the task to already cover that site
Background tasks	Unattended actions in logged-in sessions	Read-only by default; per-task site allowlist; writes and submissions queue for approval; hard step and time caps

Surface	Risk	Control
Local memory	Accumulating a profile of the user	Stores sanitized notes only, encrypted with a key in extension storage; visible and deletable in settings; never sent unless the user references it
Overlay	Page scripts reading or spoofing overlay	Shadow root is for style isolation only; the page can still cover or imitate the overlay, so it never holds secrets or approvals. Overlay text is always escaped; confirmation dialogs live in the side panel, not in the page
Guide mode	User mis-clicks on a spoofed element	Pointer targets must be DOM-backed or high-confidence vision elements; spotlight dims everything else

Prompt-injection defence is layered: a local classifier flags suspicious page text before it enters the Screen State; the planner is instructed that page content is data; server validation rejects actions outside the task’s sites; high-risk actions always need the user.

11. Performance budgets

A Tier 0 step must complete in about 1.2 s median, and the client may never block the page’s main thread for more than 50 ms. The reference device is a mid-range laptop: 4-core CPU, 8 GB RAM, integrated GPU with WebGPU (e.g. Intel Iris Xe class), Chrome stable.

11.1 Per-step latency budget (p50)

Stage	Tier 0	Tier 1	Tier 2
DOM extraction (incremental)	15 ms	15 ms	15 ms
Screenshot capture + decode	—	60 ms	60 ms
Local vision (detector + OCR + faces)	—	90 ms	90 ms

Stage	Tier 0	Tier 1	Tier 2
PII detection + redaction	20 ms	40 ms	70 ms
Egress Guard	5 ms	5 ms	40 ms (crop OCR check)
Network (India → server region, round trip)	80 ms	80 ms	150 ms
Server prefill + decode	600 ms	650 ms	650 ms + VLM 900 ms
Execute + settle	350 ms	350 ms	350 ms
Total	~1.1 s	~1.3 s	~2.3 s

Server figures assume a 7B int8 planner on one L4/A10G with prefix caching and ~80 output tokens; they must be benchmarked in M1. If they miss, the fallback is a 3B planner for simple steps with 7B on retry.

11.2 Client resource budget

Resource	Budget	Enforcement
Model download (first run)	≤ 40 MB core; voice pack ≤ 80 MB, downloaded only when voice is first enabled	CI size check on model bundle
Extra RAM, steady state	≤ 250 MB	Periodic <code>performance.measureUserAgentSpecificMemory</code> sampling in offscreen doc
Extra RAM, peak during Tier 1	≤ 350 MB	Release tensors per step; lazy-load OCR/NER
GPU memory	≤ 300 MB	One session per model; fp16/int8 only
Main-thread long tasks	None > 50 ms	All ML in worker; DOM walk chunked in idle callbacks
Idle CPU when agent not running	~0%	Models unloaded after 10 min idle; no background polling

Resource	Budget	Enforcement
Battery	No perceptible drain when idle	Same as above

11.3 Latency strategy summary

- 1. Do not look at pixels unless the DOM is insufficient (Tier 0 default).
- 2. Never send full screenshots; send text, and crops only on escalation.
- 3. Keep models warm during a task, unload when idle.
- 4. Send diffs after step 1.
- 5. Short, constrained outputs and cached prompt prefixes on the server.
- 6. Pipeline where safe: start the next DOM extraction while the server is thinking about low-risk scroll or wait actions.
- 7. Batch predictable form fills into multi-action plans (Section 8.4).

12. Evaluation and testing

Every release is gated by an automated benchmark that computes all five evaluation criteria plus the zero-leak guardrail on a fixed suite of annotated pages and end-to-end tasks.

12.1 Benchmark suite

Suite	Contents	Size (v1)	Ground truth
Perception set	Screenshots + DOM snapshots of real and synthetic pages: forms, dashboards, e-commerce, gov portals, email, video call, canvas apps	1,000 pages	Element bboxes, roles, labels, states (DOM-derived + hand-corrected for canvas/custom widgets)

Suite	Contents	Size (v1)	Ground truth
PII set	Synthetic pages with seeded PII in all forms: inputs, text, images, canvas, URLs, shadow DOM; Indian and international formats	600 pages, ~8,000 PII instances	Class + span or pixel mask per instance
Task set	Scripted end-to-end tasks on locally hosted mock sites (profile update, search + filter, multi-page form, settings change, find information)	50 tasks, 3–12 steps each	Success predicate per task, expected final state
Resource set	Fixed traces on reference laptop and a low-end laptop (WASM-only)	10 pages × 2 devices	Perf traces

Mock sites are local and deterministic so latency and success are reproducible; a smaller live-site smoke set (20 tasks) runs weekly.

12.2 Metric definitions

- **Visual context accuracy:** match predicted to GT elements by Hungarian assignment on IoU (≥ 0.5 counts). Report element recall, precision, role accuracy, label accuracy (normalized edit distance ≤ 0.2), and state accuracy.
- **PII detection:** per-class precision and recall at instance level; a text instance counts as detected if $\geq 90\%$ of its characters are covered.
- **Redaction precision:** pixel IoU between applied mask and GT mask on crops; **leak rate** = GT PII pixels or characters left visible; **over-redaction** = non-PII pixels or characters masked.
- **Client resources:** peak and steady RAM, GPU memory, main-thread long-task count, CPU time per step, from Chrome tracing and Firefox profiler.
- **End-to-end latency:** per-step p50/p95 and per-task wall time; task success rate reported alongside so latency is never optimised at the cost of success.

12.3 Test layers

1. **Unit:** PII rules (with checksum edge cases), placeholder consistency and bindings, text-to-pixel alignment (including multiline and Devanagari), coordinate transforms, schema round-trips, vault lifecycle, protocol resync rules.
2. **Model tests:** accuracy regression thresholds per model on the held-out site-family test split; ONNX outputs compared against PyTorch reference within tolerance on WebGPU and WASM.
3. **Integration, Chrome:** Playwright launches Chromium with the extension loaded (its extension support is Chromium-only) against mock sites; an intercepting proxy records all egress for canary checks.
4. **Integration, Firefox:** a separate harness loads the extension with `web-ext run` and drives it through Selenium/geckodriver (or WebDriver BiDi). Playwright's Firefox can still test the shared web code (side panel UI, overlay) without the extension.
5. **Executor tests use the real executor.** Playwright's own clicks are trusted input, so they prove nothing about our synthetic events. Tests trigger actions only through the extension's executor and adapters.
6. **Cross-browser matrix:** Chrome stable/beta, Firefox stable; WebGPU on/off.
7. **Adversarial:** prompt-injection pages, PII obfuscation pages, layout-shift pages, repeated identical controls, slow networks, dropped and duplicated protocol messages.

12.4 Data flywheel

With explicit opt-in, users can share sanitized trajectories (Screen States + actions + outcomes, already redacted). These train the planner (supervised fine-tuning on successful traces) and surface perception failures. Raw screenshots are never collected.

13. Stack, milestones and risks

The build is a TypeScript monorepo for the extension and shared schema, plus a Python service, delivered in six milestones where the privacy pipeline lands before the agent gets smart.

13.1 Tech stack

Layer	Choice
Extension	TypeScript, MV3, Vite + <code>@crxjs/vite-plugin</code> , <code>webextension-polyfill</code> , <code>web-ext</code> for Firefox
Side panel UI	Preact or React + Tailwind (small bundle)
Local inference	<code>onnxruntime-web</code> (WebGPU, WASM SIMD/threads); Transformers.js for NER tokenization
Image ops	<code>OffscreenCanvas</code> , <code>ImageBitmap</code> , WebGPU compute for resize where available
Shared protocol	JSON Schema → TS types (<code>json-schema-to-typescript</code>) and Pydantic (<code>datamodel-code-generator</code>)
Server	Python 3.12, FastAPI, Pydantic v2, Redis, vLLM (guided decoding)
Model training	PyTorch, Ultralytics (detector), Hugging Face Transformers (NER), ONNX export + <code>onnxruntime</code> quantization tools
Testing	Vitest, Playwright (Chromium extension tests), <code>web-ext</code> + Selenium/geckodriver (Firefox extension tests), mitmproxy for egress capture, pytest
Infra	Docker, one GPU node (L4/A10G), GitHub Actions CI

13.2 Repository layout

/packages	
/protocol	JSON schemas, generated TS + Python types, version tests
/extension	
/src/background	routing, capture, egress-guard
/src/offscreen	worker host (Chrome)
/src/worker	perception, fusion, pii, vault
/src/content	dom-extract, executor, mutation tracking
/src/ui	side panel / sidebar
/src/platform	chrome.ts, firefox.ts adapters
/models	training scripts, export, quantization, model cards,
hashes	
/services	
/agent-api	FastAPI app, prompts, validators, session store
/inference	vLLM configs, VLM config
/bench	
/mock-sites	deterministic test sites
/datasets	perception, PII, task suites + annotations
/harness	metric computation, reports

13.3 Milestones

Milestone	Scope	Exit criteria
M0 Foundations (1 wk)	Monorepo, protocol v0, extension shell in both browsers, mock sites	Extension loads in Chrome + Firefox; schema types generate
M1 Spikes (2 wk)	Benchmark candidate local models (incl. STT) on WebGPU/WASM; benchmark planner latency on target GPU	Measured latency/size table replaces estimates in Sections 3.4, 5 and 11
M2 Tier 0 + privacy (2 wk)	DOM extraction, L1/L2/L5 PII, vault, Egress Guard, canary tests	Zero canary leaks; Screen State for all mock pages
M3 Ask + Guide modes (2 wk)	Side panel Ask mode, overlay renderer, point / annotate , hotkey invocation	Guide completes ≥ 80% of task set with the user clicking; zero leaks

Milestone	Scope	Exit criteria
M4 Agent loop (2 wk)	Planner with constrained decoding, executor, verification, safety guards	≥ 70% task success on task set, Tier 0 only
M5 Tier 1 vision (3 wk)	Detector, OCR-on-demand, faces, icon classifier, NER, fusion	Perception and PII targets from Section 2.3 on benchmark
M6 Voice + tabs (2 wk)	Local VAD/STT/TTS, barge-in, multi-tab digests, local memory	Transcript ≤ 500 ms p50; chat-with-tabs answers cite correct tab ≥ 90%
M7 Tier 2, background, polish (2 wk)	Escalation VLM, crop redaction, diffs, multi-action plans, background tasks with approval queue, Privacy view	All v1 targets met; ≥ 85% task success; resource budgets pass on reference laptop

13.4 Risks and mitigations

Risk	Likelihood	Impact	Mitigation
WebGPU unavailable or unstable on user hardware (common on Linux, older GPUs, some Firefox versions)	High	Medium	WASM fallback path is first-class and tested; adaptive mode reduces model set
Chrome service worker lifecycle kills warm models	Medium	Medium	Host models in offscreen document, keep-alive only during active tasks
DOM-only perception misses custom widgets	Medium	High (25% criterion)	Tier 1 vision on gaps; detector trained with DOM-as-labels data
PII recall below target on free text and images	Medium	Critical	Recall-first thresholds, layered detectors, Egress Guard, user-marked regions

Risk	Likelihood	Impact	Mitigation
7B planner too slow or too weak	Medium	High	3B/7B routing; fine-tune on trajectories; multi-action plans
Over-redaction hides info the planner needs	Medium	Medium	Typed placeholders keep structure; over-redaction metric tracked
Prompt injection from page content	Medium	High	Page text treated as data, suspicious flags, confirmations, allowlist
Site anti-automation breaks execution	Low–Medium	Medium	Realistic event sequences; fall back to <code>ask_user</code>

13.5 Open questions

- ☐ Which reference laptop and GPU exactly define the resource budget for evaluation?
- ☐ Is a small local planner (1–3B, WebGPU) worth adding for trivial steps to skip the network entirely?
- ☐ Default policy for amounts and balances: redact by default or only on user opt-in?
- ☐ Which three end-to-end demo tasks best represent the product for stakeholders?

Revision log

Date	Change	Sections
2026-09-22	Weak devices no longer drop NER silently; added restricted egress mode; guardrail restated as a benchmark result	2.4, 5.3, 6.7
2026-09-22	Fusion keeps DOM and OCR privacy evidence separately; every transmitted crop gets the full pixel pipeline	5.3, 6.1, 6.4
2026-09-22	Faces use opaque fills instead of blur and pixelation	6.3, 10.1

Date	Change	Sections
2026-09-22	Vault entries bound to task, origin, document, field and operation; website vs AI-server boundary stated	6.5, 7.4, 9.2
2026-09-22	Permission model split per feature (activeTab, frames, extra tabs, scheduled sites)	3.4.3, 4.1
2026-09-22	Executor uses per-control adapters; synthetic-event limits documented; optional trusted input via debugger API	9.2
2026-09-22	Exact value verification, unknown outcome, no replay of consequential actions, protocol resync rules	7.3, 7.6, 9.3
2026-09-22	Two-way element registry, settle detection limits, shadow-root claim, WebGPU-in-worker, Firefox test harness, capture transport and coordinate frames, text-to-pixel alignment, site-family data splits, field-aware egress matching	4.1–4.3, 5.4, 6.6, 10.2, 10.4, 12.3
2026-09-22	Crop allowlist of supported content classes and languages; QR/barcode locator and script-ID classifier; unsupported content (signatures, other scripts, document photos) masked or withheld	5.1, 5.4, 6.4, 6.8, 10.2, 10.4

Sources

- [HeyClicky product site](#): hotkey invocation, talk and agent modes, on-screen drawing, screen seen only on hotkey.
- [Perplexity Comet: what it does, costs and risks](#): sidecar, chat with tabs, background assistants, permission prompts, prompt-injection history, server-side processing.
- [Comet \(browser\)](#), [Wikipedia](#): Chromium base, platforms.
- [Defeating image obfuscation with deep learning \(McPherson, Shokri, Shmatikov, 2016\)](#): faces and text remain recognisable after blur and pixelation.
- [Chrome: activeTab](#) and [MDN: extension permissions](#): scope of temporary and host permissions.

- [MDN: Event.isTrusted](#): events from `dispatchEvent` are untrusted.
- [MDN: WeakMap](#), [W3C Resource Timing](#), [MDN: WorkerNavigator.gpu](#), [Chrome: extension messaging](#), [Playwright: Chrome extensions](#).